GatherGo

設計書

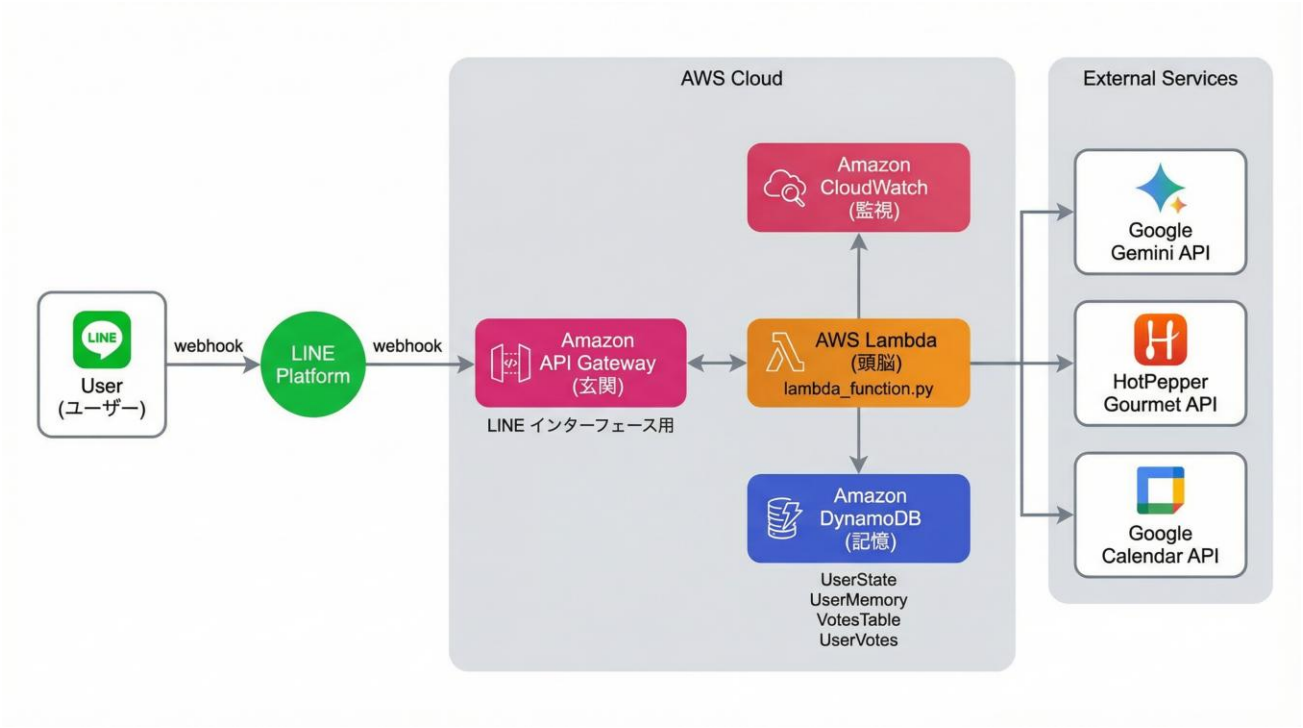
作成者	志水　長谷川　佐藤　新屋
作成日	2025/12/10
最終更新日	2026/1/15
文書番号	

麻生情報ビジネス専門学校

1. システム構成 3
1-1. システム全体像 3
1-1-1.LINE プラットフォーム(Messaging API) 3
1-1-2.AWS Lambda(バックエンドロジック) 3
1-1-3.Amazon DynamoDB(データベース) 3
1-1-4.Amazon API Gateway 3
1-1-5.外部 API(Gemini/HotPepper/Google) 3
1-2. 外部インターフェース 3
- Gemini 3
- HotPepper 3
- Google Calendar 3
2. システムのシーケンス 4
2-1 ○○機能(正常系) 4
2-2 ○○機能(異常系) 4
3. AI 機能 5
2-1. 利用する AI の一覧 5
2-2. AI の詳細 5
4. AWS Lambda 6
4-1. ファイル一覧 6
4-2. 関数一覧 6
4-3. 関数設計 7
5. その他 10
5-1. S3 バケット 10
5-2. DynamoDB 10
5-3. CloudWatch Logs 10
5-4. IAM ロール 10

1. システム構成

1-1. システム全体像



1-1-1.LINE プラットフォーム(Messaging API)

ユーザーとの対話インターフェース。メッセージの送受信や位置情報の取得を行う。

1-1-2.AWS Lambda(バックエンドロジック)

アプリケーションのメインロジック。ユーザーの意図を判断し、外部 API や DB と連携して応答を作成する。

1-1-3.Amazon DynamoDB(データベース)

NoSQL データベース。ユーザーの会話履歴、セッション状態(検索状態)、投票データなどを管理する。

1-1-4.Amazon API Gateway

Google Gemini API、HotPepper グルメ API、Google Calendar API の管理。

1-1-5.外部 API(Gemini/HotPepper/Google)

AI による自然言語処理、店舗データの検索、カレンダー情報の取得を行う外部サービス群。

1-2. 外部インターフェース

- Gemini

ユーザーからの自然言語の処理に利用

- HotPepper

店舗検索の際に使用

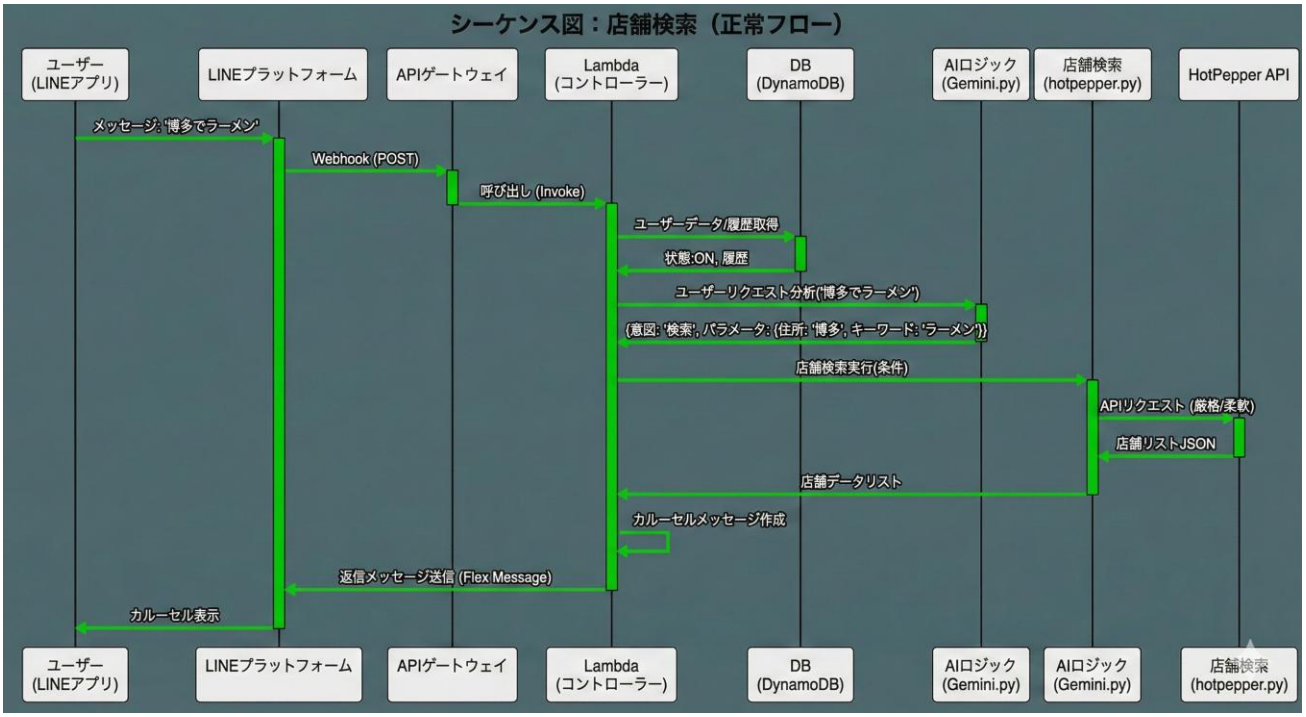
- Google Calendar

スケジュール管理の際に使用

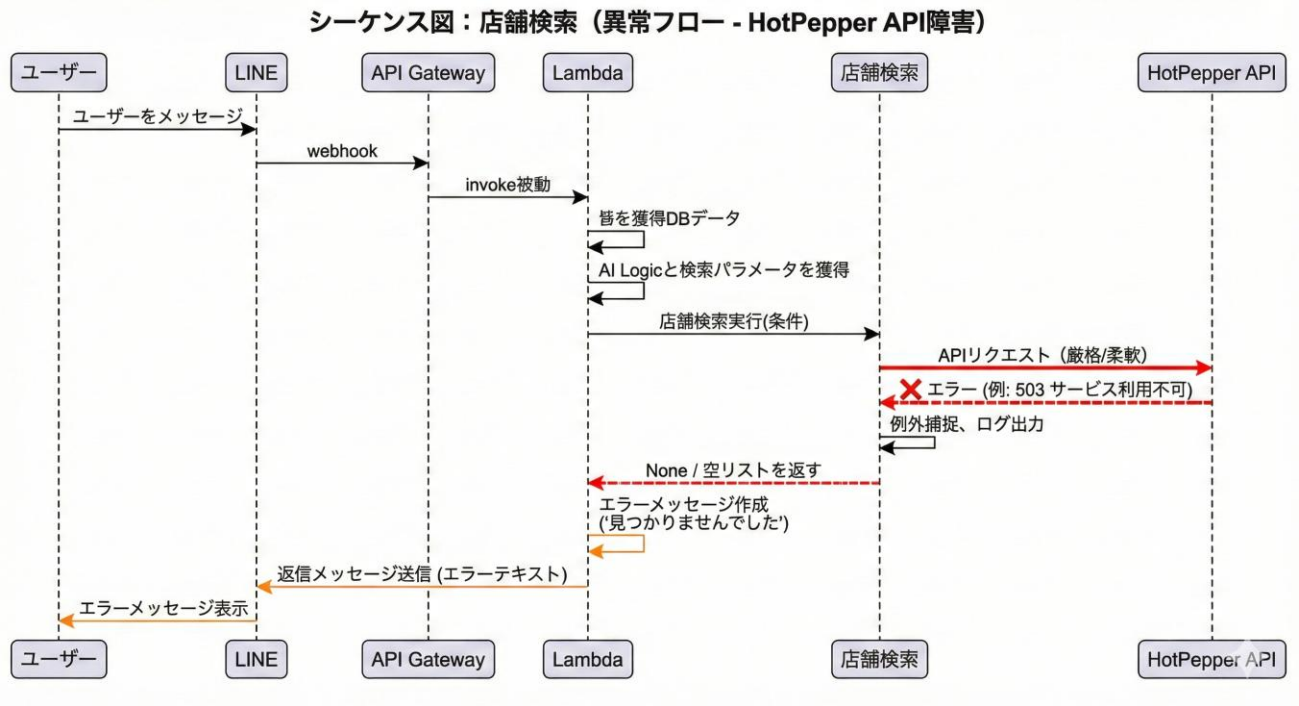
2. システムのシーケンス

代表的なユースケースごとに、シーケンス図を作成する。

2-1 チャット機能(正常系)



2-2 チャット機能(異常系)



3. AI 機能

2-1. 利用する AI の一覧

飲食店検索機能
日程投票機能
カレンダー同期機能
日程提案機能
自動予約機能

2-2. AI の詳細

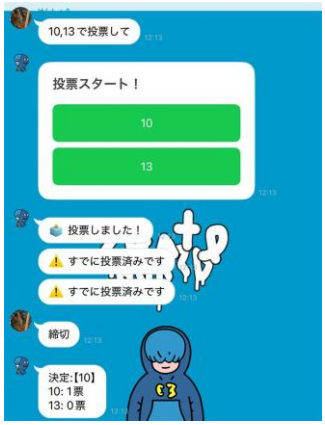
- 飲食店検索機能

LINE 上で GatherGo に「近くで居酒屋を探して」とチャットを送ると位置情報から近くの条件に合う飲食店を検索して出してくれる

※下の写真は実際の動き

- 日程投票機能

LINE 上で GatherGo に「○○日と○○日で投票して」とチャットを送ると対応した日程が投票形式になって返してくれる。※下の写真は実際の動き



- カレンダー同期機能

グループ LINE に入っているメンバーのカレンダーを同期させて予定が入っているか共有できる機能

※現在メンバーのカレンダー同期機能を開発中

- 日程提案機能

グループ LINE に入っているメンバーのカレンダーを同期させて予定の日程を立てられる機能

※現在メンバーのカレンダー同期機能を開発中

- 自動予約機能

グループ LINE 上で飲食店と日程が決まっていれば、Gather Go が対象の飲食店の予約したい日に予約

を取ってくれる
※対象機能が完成していないので開発を中断中

4. AWS Lambda

4-1. ファイル一覧

- lambda_function.py
- config.py
- requirements.txt
- .gitignore
- Gemini.py
- hotpepper.py
- google_calendar.py
- vote.py
- db.py
- utils.py
- line_client.py
- line_ui.py
- logic_plan.py
- logic_move.py
- logic_vote.py

4-2. 関数一覧

- lambda_function.py(メインコントローラー内の関数)
  - lambda_handler(event, context)
- `Gemini.py` (AI ロジック)
  - _call_gemini_api(payload)
  - analyze_user_request(user_text)
  - get_travel_cost(origin, destination, budget, keyword)
  - chat_with_gemini(user_text, history_text)
  - **`filter_shops_by_date(shops, date_text)`**
  - generate_travel_plan(place, duration, keyword)
- `hotpepper.py` (店舗検索)
  - search_hotpepper(conditions)
  - _fetch_from_api(params)
  - _get_best_budget_code(budget_value)
- `google_calendar.py` (カレンダー連携)
  - create_auth_url(user_id)
  - save_token_from_code(auth_code, user_id)

    - get_free_days(user_tokens, start_date, days)
- `vote.py` (投票システム)
    - create_vote(user_id, group_id, options)
    - add_vote(vote_id, option)
    - get_vote_result(vote_id)
    - has_user_voted(vote_id, user_id)
    - record_user_vote(vote_id, user_id, selected_option, timestamp)
- `db.py` (データベース操作)
    - get_user_data(user_id)
    - update_user_data(user_id, updates)
    - clear_user_conditions(user_id)
    - append_user_history(user_id, message, role)
    - get_history_text(user_id, limit)
    - get_user_tokens(user_id)
    - add_group_member(group_id, user_id)
    - get_group_members(group_id)
- `line_ui.py` (画面生成)
    - create_carousel_message(shops, search_keyword, ...)
    - create_location_request()
    - create_vote_flex(vote_id, options)
- `utils.py` (ユーティリティ)
    - convert_date_text(date_text)
    - _next_weekday(now, target_weekday)
- `line_client.py` (LINE 通信)
    - send_reply_message(reply_token, messages)
- `config.py` (設定)
    - get_env(key, default)
- 機能ロジックファイル (統合されている場合もありますが、ファイルとして存在)
    - handle_plan_request(params, user_data)
    - **`handle_move_request(params)`**:
    - handle_vote_request(...)

4-3. 関数設計

・関数ごとに、入力引数、戻り値、内部の処理についての設計情報をまとめる

4-3-1-1. lambda_handler

処理:

1. **Google 認証 (GET)**: カレンダー連携の認証コードを受け取り、保存処理を呼び出す。

2. **LINE Bot (POST)**: LINE からのイベント（メッセージ、ボタン操作、参加イベント）を受け取り、適切な処理（AI 解析、DB 保存、返信）に振り分ける。

4-3-2-1. _call_gemini_api(payload)
処理: Gemini API への HTTP リクエストを行う共通の内部関数。
4-3-2-2. analyze_user_request(user_text)
処理: ユーザーの発言を分析し、意図（検索、投票、計画など）とパラメータ（場所、予算など）を JSON 形式で抽出する。
4-3-2-3. get_travel_cost(origin, destination, budget, keyword)
処理: 出発地と目的地から、交通手段・費用・所要時間の目安を算出し、乗換案内のリンクと共に返す。
4-3-2-4. chat_with_gemini(user_text, history_text)
処理: 過去の会話履歴を踏まえて、ユーザーとの雑談に応答する。
4-3-2-5. filter_shops_by_date(shops, date_text)
処理: (現状は簡易実装) 営業日フィルタリング用の関数。
4-3-2-6. generate_travel_plan(place, duration, keyword)
処理: 目的地や条件をもとに、具体的な観光モデルコース（旅のしおり）を作成する。
4-3-3-1. search_hotpepper(conditions)
役割: 店舗検索のメイン関数。
特徴: 「住所検索（厳格）」と「キーワード検索（柔軟）」の両方を実行し、ヒット数が多い方を採用する「ハイブリッド検索」ロジックを実装。地名のクリーニング処理もここで行う。
4-3-3-2. _fetch_from_api(params)
処理: ホットペッパーAPI にリクエストを送り、店舗リストとヒット総数を取得する。
4-3-3-3. _get_best_budget_code(budget_value)
処理: 数値の予算（例: 3000）を、API 専用の予算コード（例: B003）に変換する。
4-3-4-1. create_auth_url(user_id)
処理: ユーザーごとの Google 認証用 URL を発行する。
4-3-4-2. save_token_from_code(auth_code, user_id)
処理: 認証後に返ってきたコードをアクセストークンに変換し、DB に保存する。
4-3-4-3. get_free_days(user_tokens, start_date, days)
処理: 指定したユーザーの向こう 1 週間分の空き日程（予定が入っていない日）をリストアップする。
4-3-5-1. create_vote(user_id, group_id, options)
処理: 新しい投票を作成し、DB に保存して投票 ID を発行する。
4-3-5-2. add_vote(vote_id, option)
処理: 指定された選択肢に票を加算する。選択肢にない項目への投票も動的に追加して受け付ける。
4-3-5-3. get_vote_result(vote_id)
処理: 投票を集計して最も多い選択肢（Winner）を決定し、投票ステータスを終了（closed）にする。
4-3-5-4. has_user_voted(vote_id, user_id)
処理: そのユーザーが既にその投票に参加済みかどうかを確認する（二重投票防止）。

4-3-5-5. record_user_vote(vote_id, user_id, selected_option, timestamp)
処理: 「誰が何に入れたか」という投票履歴を記録する。
4-3-6-1. get_user_data(user_id)
処理: データベース内のユーザーの現在の状態（ON/OFF、検索条件など）を取得する。
4-3-6-2. update_user_data(user_id, updates)
処理: データベース内のユーザーの状態を更新・保存する。
4-3-6-3. clear_user_conditions(user_id)
処理: データベース内の一時保存していた検索条件を削除する（リセット処理)。
4-3-6-4. append_user_history(user_id, message, role)
処理:会話履歴をデータベース内のリストに追加保存する。
4-3-6-5. get_history_text(user_id, limit)
処理: 過去の会話履歴を AI が読みやすいテキスト形式で取得する。

4-3-6-6. get_user_tokens(user_id)
処理: カレンダー連携用のトークンを取得する。
4-3-6-7. add_group_member(group_id, user_id)
処理: グループ内で発言や操作をしたユーザーをメンバーリストに追加する。
4-3-6-8. get_group_members(group_id)
処理: グループに参加しているメンバーID のリストを取得する。
4-3-7-1. create_carousel_message(shops, search_keyword, ...)
処理: 検索された店舗リストを、横スクロール可能なカード形式（カルーセル）のメッセージデータに変換する。
4-3-7-2. create_location_request()
処理: ユーザーに位置情報の送信を促すクイックリプライボタンを作成する。
4-3-7-3. create_vote_flex(vote_id, options)
処理: 投票ボタンが付いた選択肢パネル（Flex Message）を作成する。
4-3-8-1. convert_date_text(date_text)
処理:「明日」「今度の土曜」といった言葉を、具体的な日付（MM/DD 形式）に変換する。
4-3-8-2. _next_weekday(now, target_weekday)
処理: 指定した曜日が来る日付を計算する。
4-3-9-1. send_reply_message(reply_token, messages)
処理: LINE Messaging API にリクエストを送り、ユーザーへ返信を届ける。
4-3-10-1. get_env(key, default)
役割: 環境変数（API キーなど）を安全に読み込むヘルパー関数。

5. その他

5-1. S3 バケット

- Lambda 関数のデプロイ用アーティファクト（ZIP ファイル）の保存領域として使用。

5-2. DynamoDB

- **GatherGoUserState**: ユーザーの状態（ON/OFF、検索条件）を管理。
- **GatherGoUserMemory**: 直近の会話ログを保存。
- **GatherGo_VotesTable**: 投票の選択肢と集計結果を管理。
- **GatherGo_UserVotes**: ユーザーごとの投票済みフラグを管理。

5-3. CloudWatch Logs

- Lambda 関数の実行ログおよびエラートレースの出力先として使用。

5-4. IAM ロール

- Lambda 関数に対し、DynamoDB へのアクセス権限およびログ出力権限を付与する。